



Singularry SyrupLoopFacet

AUDIT REPORT

Version 2.0.0

Serial No. 2026052000012020

Presented by Fairyproof

May 20, 2026

01. Introduction

This document includes the results of the audit performed by the Fairyproof team on the Singularry SyrupLoopFacetV2 project.

Audit Start Time:

May 11, 2026

Audit End Time:

May 20, 2026

Audited Code's Github Repository:

<https://github.com/singularry/slywallet-contracts-foundry/>

Audited Code's Github Commit Number When Audit Started:

270c67b017367df55494e25478d93891e4f32cfb

Audited Code's Github Commit Number When Audit Ended:

ee378d78b8696164dfc240d65cdbca5f270dee90

Audited Source Files:

The source files audited include all the files as follows:

```
src/slywallet/facets/syruploop
├── libraries
│   ├── LibSyrupLoopCore.sol
│   └── LibSyrupLoopMath.sol
├── SLYWalletSyrupLoopExecutionFacet.sol
├── SLYWalletSyrupLoopManagementFacet.sol
├── SLYWalletSyrupLoopMonitoringFacet.sol
├── SLYWalletSyrupLoopStorage.sol
├── SyrupLoopErrors.sol
├── SyrupLoopEvents.sol
└── SyrupLoopTypes.sol

2 directories, 9 files
```

The goal of this audit is to review Singularry's solidity implementation for its SyrupLoopFacetV2 function, study potential security vulnerabilities, its general design and architecture, and uncover bugs that could compromise the software in production.

We make observations on specific areas of the code that present concrete problems, as well as general observations that traverse the entire codebase horizontally, which could improve its quality as a whole.

This audit only applies to the specified code, software or any materials supplied by the Singularry team for specified versions. Whenever the code, software, materials, settings, environment etc is changed, the comments of this audit will no longer apply.

— Disclaimer

Note that as of the date of publishing, the contents of this report reflect the current understanding of known security patterns and state of the art regarding system security. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. If the audited source files are smart contract files, risks or issues introduced by using data feeds from offchain sources are not extended by this review either.

Given the size of the project, the findings detailed here are not to be considered exhaustive, and further testing and audit is recommended after the issues covered are fixed.

To the fullest extent permitted by law, we disclaim all warranties, expressed or implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We do not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

— Methodology

The above files' code was studied in detail in order to acquire a clear impression of how the its specifications were implemented. The codebase was then subject to deep analysis and scrutiny, resulting in a series of observations. The problems and their potential solutions are discussed in this document and, whenever possible, we identify common sources for such problems and comment on them as well.

The Fairyproof auditing process follows a routine series of steps:

1. Code Review, Including:

- Project Diagnosis

Understanding the size, scope and functionality of your project's source code based on the specifications, sources, and instructions provided to Fairyproof.

- Manual Code Review

Reading your source code line-by-line to identify potential vulnerabilities.

- Specification Comparison

Determining whether your project's code successfully and efficiently accomplishes or executes its functions according to the specifications, sources, and instructions provided to Fairyproof.

2. Testing and Automated Analysis, Including:

- Test Coverage Analysis

Determining whether the test cases cover your code and how much of your code is exercised or executed when test cases are run.

- Symbolic Execution

Analyzing a program to determine the specific input that causes different parts of a program to execute its functions.

3. Best Practices Review

Reviewing the source code to improve maintainability, security, and control based on the latest established industry and academic practices, recommendations, and research.

— Structure of the document

This report contains a list of issues and comments on all the above source files. Each issue is assigned a severity level based on the potential impact of the issue and recommendations to fix it, if applicable. For ease of navigation, an index by topic and another by severity are both provided at the beginning of the report.

— Documentation

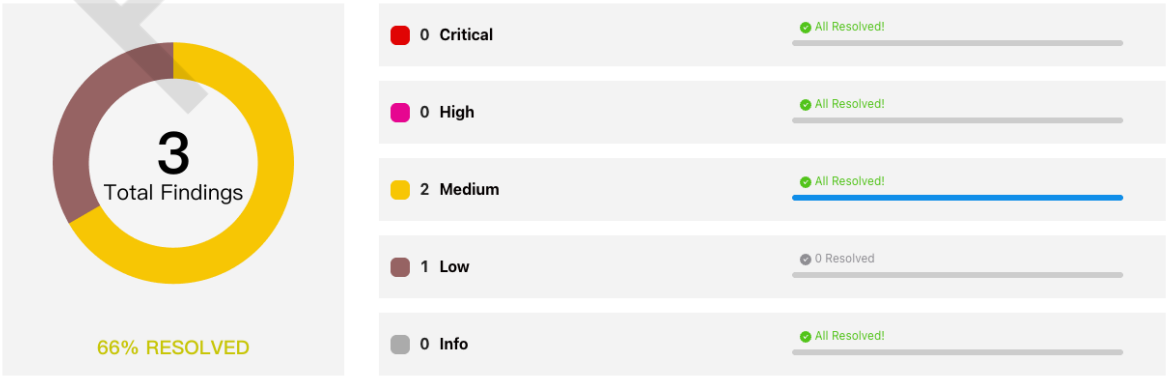
For this audit, we used the following source(s) of truth about how the token issuance function should work:

- Website:<https://www.singularry.org/>
- Whitepaper:<https://docs.singularry.org/>
- Source Code: <https://github.com/singularry/slywallet-contracts-foundry/>

These were considered the specification, and when discrepancies arose with the actual code behavior, we consulted with the Singularry team or reported an issue.

— Comments from Auditor

Serial Number	Auditor	Audit Time	Result
2026052000012020	Fairyproof Security Team	May 11, 2026 - May 20, 2026	Low Risk



Summary:

The Fairyproof security team used its auto analysis tools and manual work to audit the project. During the audit, two issues of medium-severity and one issue of low-severity were uncovered. The Singularry team fixed two issues of medium, and acknowledged the remaining issues.

02. About Fairyproof

[Fairyproof](#) is a leading technology firm in the blockchain industry, providing consulting and security audits for organizations. Fairyproof has developed industry security standards for designing and deploying blockchain applications.

03. Introduction to Singularity

Singularity is a platform of creating next-generation financial systems with intelligent, autonomous protocols and seamless cross-chain operations.

The above description is quoted from relevant documents of Singularity.

04. Major functions of audited code

SyrupLoopFacetV2 is an enhanced and optimized version of its V1. Compared to the original version, it now supports multiple Fluid Vaults, allows Native Tokens as collatera or borrowed assets, and enables swaps via PancakeV3.

05. Coverage of issues

The issues that the Fairyproof team covered when conducting the audit include but are not limited to the following ones:

- Access Control
- Admin Rights
- Arithmetic Precision
- Code Improvement
- Contract Upgrade/Migration
- Delete Trap
- Design Vulnerability
- DoS Attack
- EOA Call Trap
- Fake Deposit
- Function Visibility
- Gas Consumption
- Implementation Vulnerability

- Inappropriate Callback Function
- Injection Attack
- Integer Overflow/Underflow
- IsContract Trap
- Miner's Advantage
- Misc
- Price Manipulation
- Proxy selector clashing
- Pseudo Random Number
- Re-entrancy Attack
- Replay Attack
- Rollback Attack
- Shadow Variable
- Slot Conflict
- Token Issuance
- Tx.origin Authentication
- Uninitialized Storage Pointer

06. Severity level reference

Every issue in this report was assigned a severity level from the following:

Critical severity issues need to be fixed as soon as possible.

High severity issues will probably bring problems and should be fixed.

Medium severity issues could potentially bring problems and should eventually be fixed.

Low severity issues are minor details and warnings that can remain unfixed but would be better fixed at some point in the future.

Informational is not an issue or risk but a suggestion for code improvement.

07. Major areas that need attention

Based on the provided source code the Fairyproof team focused on the possible issues and risks related to the following functions or areas.

- Function Implementation

We checked whether or not the functions were correctly implemented.

We found some issues, for more details please refer to [FP-1,FP-2,FP-3] in "09. Issue description".

- Access Control

We checked each of the functions that could modify a state, especially those functions that could only be accessed by owner or administrator

We didn't find issues or risks in these functions or areas at the time of writing.

- Token Issuance & Transfer

We examined token issuance and transfers for situations that could harm the interests of holders.

We didn't find issues or risks in these functions or areas at the time of writing.

- State Update

We checked some key state variables which should only be set at initialization.

We didn't find issues or risks in these functions or areas at the time of writing.

- Asset Security

We checked whether or not all the functions that transfer assets were safely handled.

We didn't find issues or risks in these functions or areas at the time of writing.

- Miscellaneous

We checked the code for optimization and robustness.

We didn't find issues or risks in these functions or areas at the time of writing.

08. List of issues by severity

Index	Title	Issue/Risk	Severity	Status
FP-1	Redundant debtToken during the <code>unwind</code> operation.	Design Vulnerability	Low	Acknowledged
FP-2	Initial executing fund is inconsistent with the setting	Design Vulnerability	Medium	✓ Fixed
FP-3	Fee collection and calculation are inaccurate	Design Vulnerability	Medium	✓ Fixed

09. Issue descriptions

[FP-1] Redundant debtToken during the unwind operation.

Design Vulnerability

Low

Acknowledged

Issue/Risk: Design Vulnerability

Description:

Since it is a proportional calculation of the withdrawable collateral assets, not the exact calculation of the required collateral assets, there will be a small amount of redundant debt assets after swapping collateral assets into debt assets and paying the debt when it is not completely closed. But the redundant debt assets can be used for debt repayment at the next closing. The returned assets upon full closed is exactly debt assets, so there is no redundancy.

Recommendation:

It is necessary to inform users of the relevant matters and recommend that they completely close the position.

Update/Status:

The team has acknowledged the issue.

[FP-2] Initial executing fund is inconsistent with the setting

Design Vulnerability

Medium

✓ Fixed

Issue/Risk: Design Vulnerability

Description:

In the `syrupLoopExecute` function, when executing the loop, the code is as follows: `uint256 debtForLoop = IERC20(config.debtToken).balanceOf(address(this));` It used token balances of the wallet instead of `initialAmount` for the executing, which led to unexpected behaviors such as costing all debt assets of the wallet into the strategy. In addition, the fees are also calculated based on the `initialAmount`, which will lead to inaccurate calculation of the fees.

Recommendation:

A new field 'nextAmountForLoop' is added to the 'Position' structure to record the amount of assets returned in each loop, preventing all assets in the wallet from being invested. Use code like this in the loop: `uint256 debtForLoop = pos.nextAmountForLoop;`

Update/Status:

The team has fixed the issue.

[FP-3] Fee collection and calculation are inaccurate

Design Vulnerability

Medium

✓ Fixed

Issue/Risk: Design Vulnerability

Description:

In the current implementation, fees are based on comparing the `debtToken` returned per close execution to the initial amount, rather than the cumulative return value compared to the initial amount. This causes the cumulative return value to exceed the initial quantity after multiple close executions, but the return value each time is less than the initial amount, resulting in zero fees.

Recommendation:

Add a field `accumulatedReturnedDebt` to `Position` to record the accumulated returned `debtToken`, so that it is correctly compared with the initial amount.

Update/Status:

The team has fixed the issue.

10. Recommendations to enhance the overall security

We list some recommendations in this section. They are not mandatory but will enhance the overall security of the system if they are adopted.

- N/A

11. Appendices

11.1 Unit Test

1. UnitTestOutput.md

```
Ran 11 tests for
test/slywallet/syruploop/SyrupLoopManageFacet.t.sol:SyrupLoopManageFacetTest
[PASS] testCreatePositionWithNewVault() (gas: 1673544)
[PASS] testDeactivateVault() (gas: 286343)
[PASS] testRegisterFluidVault() (gas: 291652)
[PASS] testRegisterMultipleVaults() (gas: 491986)
[PASS] testSyrupLoopCreatePositionLegacy() (gas: 848108)
[PASS] testSyrupLoopInitialize() (gas: 70292)
[PASS] testSyrupLoopPause() (gas: 42118)
[PASS] testSyrupLoopSetFeeConfig() (gas: 150348)
```

```
[PASS] testSyrupLoopSetSafetyParams() (gas: 83423)
[PASS] testSyrupLoopSetSwapSlippage() (gas: 42707)
[PASS] testSyrupSetAndGetFeeConfig() (gas: 150472)
Suite result: ok. 11 passed; 0 failed; 0 skipped; finished in 2.73s (6.69ms CPU time)

Ran 1 test for test/slywallet/syruploop/SyrupLoopExeFacet.t.sol:SyrupLoopExeFacetTest
[PASS] test_strategy_ETH_BTCB_via_PancakeSwap() (gas: 2313014)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.73s (9.44ms CPU time)

Ran 6 tests for
test/slywallet/syruploop/Syrup_BNB_BTCB_Vault.t.sol:BNB_BTCB_Vault_SyrupLoopTest
[PASS] testCancelBNB_BTCB_Position() (gas: 674077)
[PASS] testCloseBNB_BTCB_Position() (gas: 4169916)
[PASS] testDeleverageBNB_BTCB_PositionWithDebt() (gas: 2685993)
[PASS] testDeleverageBNB_BTCB_PositionWithoutDebt() (gas: 2957126)
[PASS] testEmergencyExitBNB_BTCB_Position() (gas: 2662617)
[PASS] testExecuteBNB_BTCB_Position() (gas: 2694546)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 2.75s (62.77ms CPU time)

Ran 6 tests for
test/slywallet/syruploop/Syrup_ETH_BNB_Vault.t.sol:ETH_BNB_Vault_SyrupLoopTest
[PASS] testCancelETH_BNB_Position() (gas: 625142)
[PASS] testCloseETH_BNB_Position() (gas: 4113077)
[PASS] testDeleverageETH_BNB_PositionWithDebt() (gas: 2660921)
[PASS] testDeleverageETH_BNB_PositionWithoutDebt() (gas: 2927040)
[PASS] testEmergencyExitETH_BNB_Position() (gas: 2630222)
[PASS] testExecuteETH_BNB_Position() (gas: 2678377)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 2.75s (62.04ms CPU time)

Ran 9 tests for
test/slywallet/syruploop/Syrup_Eth_BTCB_Vault.t.sol:Eth_BTCB_Vault_SyrupLoopTest
[PASS] testCancelETH_BTCB_Position() (gas: 275360)
[PASS] testCloseETH_BTCB_Position() (gas: 3649886)
[PASS] testDeleverageETH_BTCB_PositionWithDebt() (gas: 2313919)
[PASS] testDeleverageETH_BTCB_PositionWithInsufficientDebt() (gas: 2579202)
[PASS] testDeleverageETH_BTCB_PositionWithoutDebt() (gas: 2603840)
[PASS] testETH_BTCBEmergencyExit() (gas: 2252503)
[PASS] testExecuteETH_BTCB_Position() (gas: 2130826)
[PASS] testExecuteUnexistedPosition() (gas: 28742)
[PASS] testRebalanceETH_BTCB_Position() (gas: 3398372)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 2.75s (83.87ms CPU time)

Ran 7 tests for
test/slywallet/syruploop/SyrupUSDT_USDT_Vault.t.sol:SyrupUSDT_USDT_Vault_SyrupLoopTest
[PASS] testCancelSyrupUSDT_USDT_Position() (gas: 898074)
[PASS] testCloseSyrupusdt_Usdt_Position() (gas: 3913583)
[PASS] testDeleverageSyrupusdt_Usdt_PositionWithDebt() (gas: 2624743)
[PASS] testDeleverageSyrupusdt_Usdt_PositionWithoutDebt() (gas: 2888679)
[PASS] testEmergencyExitSyrupusdt_Usdt_Position() (gas: 2617801)
[PASS] test_ExecuteSyrupusdt_Usdt_Position() (gas: 2607991)
[PASS] test_ExecuteSyrupusdt_Usdt_PositionOld() (gas: 2599928)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 4.44s (69.47ms CPU time)

Ran 6 test suites in 4.45s (18.14s CPU time): 40 tests passed, 0 failed, 0 skipped (40
total tests)
```

11.2 External Functions Check Points

1. CheckPoint.md

File:

src/slywallet/facets/syruploop/SLYWalletSyrupLoopExecutionFacet.sol

contract: SLYWalletSyrupLoopExecutionFacet is SyrupLoopEvents

(Custom interfaces exclude members inherited from third-party dependencies)

Index	Custom Interface	StateMutability	Modifier	UnitTest	Miscellaneous
1	syrupLoopExecute(uint256, uint256)		whenInitialized whenNotPaused nonReentrant	Passed	Anyone can call
2	syrupLoopRebalance(uint256, uint256)		onlyAdmin whenInitialized whenNotPaused nonReentrant	Passed	
3	syrupLoopDeleverage(uint256, uint256)		onlyAdmin whenInitialized nonReentrant	Passed	
4	syrupLoopCheckRebalanceNeeded(uint256)	view		Passed	
5	syrupLoopGetRemainingLoops(uint256)	view		Passed	

File:

src/slywallet/facets/syruploop/SLYWalletSyrupLoopManagementFacet.sol

contract: SLYWalletSyrupLoopManagementFacet is SyrupLoopEvents

(Custom interfaces exclude members inherited from third-party dependencies)

Index	Custom Interface	StateMutability	Modifier	UnitTest	Miscellaneous
1	syrupLoopInitialize(address, address, address, address, uint256)		onlyAdmin	Passed	Only Once
2	fluidLoopRegisterVault(FluidVaultConfig)		onlyAdmin whenInitialized	Passed	
3	fluidLoopDeactivateVault(bytes32)		onlyAdmin whenInitialized	Passed	
4	fluidLoopGetVaultConfig(bytes32)	view			
5	fluidLoopGetRegisteredVaults()	view			
6	fluidLoopCreatePosition(FluidLoopCreateParams)	payable	onlyAdmin whenInitialized whenNotPaused nonReentrant	Passed	
7	syrupLoopCreatePosition(SyrupCreateParams)		onlyAdmin whenInitialized whenNotPaused nonReentrant	Passed	
8	syrupLoopClosePosition(SyrupCloseParams)		onlyAdmin whenInitialized nonReentrant	Passed	
9	syrupLoopEmergencyExit(uint256)		onlyAdmin whenInitialized nonReentrant	Passed	
10	syrupLoopCancelPosition(uint256)		onlyAdmin whenInitialized nonReentrant	Passed	
11	syrupLoopSetSafetyParams(uint256, uint256, uint256, uint256, uint256, uint256)		onlyAdmin whenInitialized	Passed	
12	syrupLoopGetRebalanceCooldown()	view			
13	syrupLoopGetPegDiscount()	view			
14	syrupLoopSetSwapSlippage(uint256)		onlyAdmin whenInitialized	Passed	
15	syrupLoopSetMinLoopAmount(uint256)		onlyAdmin whenInitialized	Passed	
16	syrupLoopGetSwapSlippage()	view			
17	syrupLoopGetActivePosition()	view			
18	syrupLoopSetFeeConfig(uint256, uint256, address)		onlyAdmin whenInitialized	Passed	
19	syrupLoopPause()		onlyAdmin whenInitialized	Passed	
20	syrupLoopUnpause()		onlyAdmin whenInitialized	Passed	
21	syrupLoopAdminResetActivePosition()		onlyAdmin	Passed	
22	syrupLoopGetPosition(uint256)	view	returns (SLYWalletSyrupLoopStorage.LoopPosition memory)		
23	syrupLoopGetConfig()	view			
24	syrupLoopIsInitialized()	view			
25	syrupLoopIsPaused()	view			
26	syrupLoopGetUserPositions(address)	view			
27	syrupLoopGetFeeConfig()	view			

File:

src/slywallet/facets/syruploop/SLYWalletSyrupLoopMonitoringFacet.sol

contract: SLYWalletSyrupLoopMonitoringFacet

(Custom interfaces exclude members inherited from third-party dependencies)

Index	Custom Interface	StateMutability	Modifier	UnitTest	Miscellaneous
1	syrupLoopHealthCheck(uint256)	view	whenInitialized		
2	syrupLoopGetEffectiveLeverage(uint256)	view	whenInitialized		
3	syrupLoopGetNetAPY(uint256, uint256, uint256)	view	whenInitialized		
4	syrupLoopGetUnrealizedPnL(uint256)	view	whenInitialized		
5	syrupLoopGetDexPriceGain(uint256)	view	whenInitialized		
6	syrupLoopGetOptimalLoops(uint256)	view	whenInitialized		
7	syrupLoopGetDexPrice()	view	whenInitialized		



<https://medium.com/@FairyproofT>



<https://twitter.com/FairyproofT>



<https://www.linkedin.com/company/fairyproof-tech>



https://t.me/Fairyproof_tech



Reddit: <https://www.reddit.com/user/FairyproofTech>

